Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251230**
Nama Lengkap	**Okky Alexander**
Minggu ke / Materi	**09 / Pengolahan String dan Regular Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI PENGANTAR STRING

String merupakan sekumpulan karakter yang membentuk satu kesatuan untuk menyimpan teks, baik berupa kata maupun kalimat dalam program komputer. Secara teknis, data ini disimpan menggunakan pengkodean ASCII. Tidak semua bahasa pemrograman menyediakan String sebagai tipe data bawaan; contohnya bahasa C yang menganggap String sebagai kumpulan karakter (array of characters). Oleh karena itu, String dikategorikan sebagai tipe data bentukan karena mampu menampung banyak nilai karakter sekaligus dalam satu variabel.

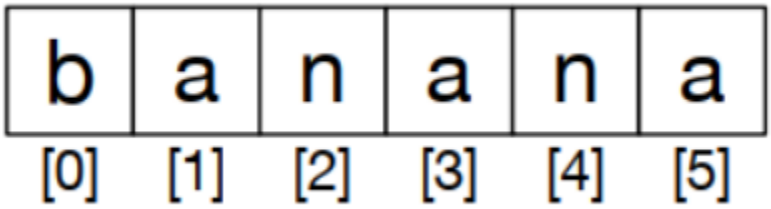
MATERI PENGAKSESAN STRING DAN MANIPULASI STRING

Proses pembuatan String diawali dengan mendeklarasikan variabel yang langsung diisi dengan nilai tertentu. Data String dapat dikelola secara menyeluruh melalui nama variabelnya, atau diakses tiap elemennya berdasarkan posisi indeks karakter tersebut. Sistem indeks ini mengikuti aturan *list*, yaitu dimulai dari angka 0. Sangat penting untuk diingat bahwa nilai indeks harus berupa bilangan bulat, karena sistem tidak mendukung penggunaan bilangan pecahan untuk menentukan posisi karakter.

Jika kamu punya namasaya = "Antonius", maka:

- namasaya[0] → Menghasilkan huruf **'A'**.
- namasaya[1] → Menghasilkan huruf **'n'**.

Di dalam memori komputer, **string** dikelola secara sekuensial dalam bentuk daftar (*list*) karakter. Setiap karakter diatur secara berurutan dengan sistem pengindeksan yang dimulai dari angka **nol**. Sebagai ilustrasi, pada Gambar 9.1, kata **"banana"** direpresentasikan dalam memori sebagai deretan elemen yang masing-masing menempati posisi indeks tertentu secara berurutan.



Gambar 9.1: Bentuk string di dalam memory

MATERI OPERATOR DAN METODE STRING

OPERATOR in

Dalam pengolahan String, keberadaan suatu sub-kalimat (*substring*) di dalam kalimat lain dapat divalidasi menggunakan operator in. Operasi ini akan memberikan luaran berupa nilai logika, yaitu True jika *substring* ditemukan, atau False jika tidak. Selain pengecekan keberadaan teks, String juga mendukung

operasi perbandingan (*comparison*) yang fungsinya sama-sama menghasilkan nilai Boolean (True atau False).

FUNGSI len

Fungsi len() digunakan untuk mengukur panjang suatu string atau menghitung total karakter yang terkandung di dalamnya. Untuk mengakses karakter paling akhir, kita perlu merujuk pada indeks ke-len(string) - 1. Hal ini dikarenakan sistem penomoran indeks dimulai dari angka 0. Selain metode tersebut, Python juga menyediakan cara yang lebih ringkas untuk mengambil karakter dari posisi belakang menggunakan indeks negatif, seperti [-1] untuk karakter terakhir dan [-2] untuk karakter kedua dari belakang.

TRAVERSING STRING

Traversing String adalah proses mengakses karakter di dalam string secara satu per satu. Untuk menampilkan string huruf demi huruf, kita dapat menggunakan mekanisme perulangan (*looping*) dengan dua metode utama:

1. Melalui Akses Indeks: Menggunakan perulangan (seperti while) dengan memanfaatkan variabel pencacah sebagai penunjuk posisi indeks karakter.
2. Tanpa Akses Indeks (Otomatis): Menggunakan perulangan (seperti for-in) yang secara otomatis mengambil setiap elemen karakter langsung dari dalam string tanpa perlu mendefinisikan indeks secara manual.

STRING SLICE

String Slicing adalah teknik untuk mengambil potongan tertentu (*substring*) dari sebuah string dengan menentukan rentang indeks, mulai dari posisi awal hingga sebelum posisi akhir (indeks akhir-1). Sintaksis yang digunakan adalah [awal:akhir], di mana pengisian nilai awal atau akhir bersifat opsional.

Penting untuk dipahami bahwa String bersifat immutable, artinya isi karakter di dalamnya tidak dapat dimodifikasi secara langsung setelah dideklarasikan. Jika ingin mengubah isi sebuah string, kita harus membuat variabel baru untuk menampung hasil modifikasi tersebut, karena mengubah elemen indeks secara langsung akan memicu kesalahan (*TypeError*).

Berikut adalah beberapa method String yang sering digunakan:

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Ingat, string itu bersifat immutable (isinya permanen). Jadi, *method* apa pun yang kamu jalankan akan memberikan hasil berupa string baru tanpa mengotak-atik string yang asli. Daftar lengkap fungsi-fungsi ini bisa kamu cek langsung di dokumentasi resmi Python.

Operator * dan + pada String

Dalam bahasa pemrograman Python, operator aritmatika seperti + dan * memiliki fungsi khusus saat diterapkan pada String. Operator + berfungsi untuk melakukan konkatenasi atau penggabungan dua string menjadi satu kesatuan teks. Sementara itu, operator * digunakan untuk replikasi, yaitu menduplikasi atau menampilkan string secara berulang sesuai dengan jumlah pengali yang ditentukan.

MATERI PARSING STRING

Parsing String merupakan teknik untuk menganalisis dan mengurai sebuah string menjadi bagian-bagian yang lebih kecil (token) guna menemukan, mengambil, atau memodifikasi informasi spesifik. Sebagai contoh, jika kita memiliki kalimat yang mengandung data tanggal, kita dapat melakukan parsing untuk mengubah format tanggal tersebut.

Prosesnya melibatkan beberapa tahapan sistematis:

1. **Tokenisasi:** Memecah kalimat menjadi daftar kata berdasarkan pemisah spasi.
2. **Identifikasi:** Menyeleksi token yang diawali oleh karakter angka.

3. **Sub-Parsing:** Melakukan pemecahan ulang pada token tanggal menggunakan pemisah tanda hubung (-).
4. **Rekonstruksi:** Menyusun kembali komponen tanggal (bulan, hari, tahun) ke dalam format baru yang diinginkan.

MATERI PENGANTAR REGEX

Setelah mempelajari berbagai cara standar untuk mengolah string, seringkali kita menemui kesulitan saat harus menangani kasus yang lebih kompleks. Untuk itulah kita memerlukan teknik yang lebih bertenaga dan efisien, yaitu **Regular Expression** atau biasa disebut **Regex**.

Secara definisi, **Regex** adalah sebuah ekspresi berbentuk pola karakter yang dirancang untuk mendeteksi kesesuaian pola teks di dalam string lain. Fungsi utama Regex adalah membantu kita dalam:

- **Pencarian:** Menemukan bagian teks yang spesifik berdasarkan pola.
- **Penggantian:** Menukar bagian string dengan teks lain sesuai kriteria.
- **Penghapusan:** Menghilangkan elemen teks tertentu.

Intinya, Regex mempermudah proses **parsing string** yang biasanya cukup merepotkan jika hanya menggunakan perintah dasar seperti split() atau find(). Meskipun kemampuannya sangat hebat dalam mencari dan menarik data (*extracting*), Regex memiliki aturan penulisan pola yang tergolong cukup rumit.

Tidak semua bahasa pemrograman menyediakan dukungan bawaan untuk ini, namun **Python** adalah salah satu yang mendukungnya secara penuh melalui pustaka khusus. Kita cukup melakukan import re untuk mengakses fiturnya, di mana fungsi search() menjadi salah satu alat yang paling mudah dan umum digunakan untuk memulai.

MATERI META CHARACTER, ESCAPED CHARACTER, SET OF CHARACTER, DAN FUNGSI REGEX PADA LIBARY PYTHON

Dalam menyusun pola **Regular Expression**, kita menggunakan simbol-simbol khusus yang dikenal sebagai **Meta Character**. Simbol ini berfungsi sebagai aturan main untuk mendefinisikan kriteria pencarian yang lebih fleksibel dan spesifik. Tanpa memahami karakter-karakter ini, kita hanya bisa mencari teks yang sifatnya statis (sama persis).

Beberapa kegunaan utama dari Meta Character ini antara lain:

- **Menentukan Posisi:** Mengatur apakah pencarian harus dimulai dari awal kalimat atau diakhiri pada bagian tertentu.
- **Menentukan Jumlah:** Mengatur berapa kali sebuah karakter boleh muncul (misalnya: sekali, berkali-kali, atau tidak sama sekali).
- **Pengelompokan:** Menentukan pilihan karakter tertentu (misalnya: hanya angka, hanya huruf kecil, atau karakter simbol saja).

Dengan menguasai daftar karakter spesial yang ada pada **Tabel 9.3**, kita dapat membuat pola pencarian yang sangat kompleks, seperti memvalidasi format email, nomor telepon, atau memisahkan data teks yang sangat berantakan secara otomatis.

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Gambar 9.3: Special Character pada Python

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Gambar 9.4: Escaped Character pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Gambar 9.5: Himpunan Karakter pada Regex

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Gambar 9.6: Fungsi Regex pada Python

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 9.1

```
import re

def jancok(a,b):
    if sorted(re.sub(r"\W+", "", a.lower())) == sorted(re.sub(r"\W+", "", b.lower())):
        print(f"{a}, {b} ini Anagram")
    else:
        print("Bukan anagram")

jancok("mata", "atma")
jancok("mata", "tama")
jancok("mata" , "maat")
jancok("mata" , "taam")
```

Gambar 9.7: Program pendeteksi anagram

Program bertujuan untuk mendeteksi apakah dua kata adalah anagram atau tidak. Caranya, kata pertama dan kata kedua dibersihkan dari karakter selain huruf menggunakan regex \W+, lalu semuanya diubah menjadi huruf kecil agar seragam. Setelah itu huruf-huruf dalam kedua kata diurutkan dan dibandingkan; jika sama, maka keduanya adalah anagram

LATIHAN 9.2

```
import re

def count_word_occurrences(kal, kat):
    pola = r'\b' + re.escape(kat) + r'\b'

    ksat = re.findall(pola, kal, flags=re.IGNORECASE)
    return len(ksat)

kalimat = "Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan"
kata_dicari = "makan"
jumlah = count_word_occurrences(kalimat, kata_dicari)

print(f"{kata_dicari} ada {jumlah} buah")
```

Gambar 9.8: Program menghitung frekuensi kemunculan kata

Pada Latihan 9.2, program menghitung frekuensi kemunculan sebuah kata dalam kalimat menggunakan regex. Dengan pola \bword\b, program mencari kata target sebagai kata utuh, sehingga tidak tertangkap bagian dari kata lain, dan menghitung banyaknya hasil pencarian, sambil mengabaikan besar kecil huruf menggunakan re.IGNORECASE

LATIHAN 9.3

```
import re

def jamcikk(a):
    return re.sub(r"\s+", " ", a).strip()

kalimat ="saya            tidak          suka        memancing          ikan"

print(jamcikk(kalimat))
```

Gambar 9.9: Program menghapus spasi berlebih

Untuk Latihan 9.3, program diminta untuk menghapus spasi berlebih dalam sebuah string, sehingga semua spasi berturut-turut diubah menjadi satu spasi menggunakan regex \s+, kemudian hasil akhirnya dirapikan dengan menghapus spasi di awal dan akhir string

LATIHAN 9.4

```
import re

def cekombak(n):
    hasil = re.findall(r"\w+",n)
    pendek = min(hasil, key=len)
    panjang = max(hasil, key=len)
    return panjang, pendek

kalimat = "red snakes and a black frog in the pool"

p, pe = cekombak(kalimat)

print(f" terpendek: {pe},terpanjang: {p}")
```

Gambar 9.9: Program mencari kata terpendek dan terpanjang

Latihan 9.4, program mencari kata terpendek dan terpanjang dalam sebuah kalimat. Semua kata diambil menggunakan regex \w+, lalu dibandingkan panjangnya untuk menemukan mana yang paling pendek dan mana yang paling panjang.

LATIHAN 9.5

```python
import re
from datetime import datetime

def nemutgldnhtg(k):
    tgl = re.findall(r"\d{4}-\d{2}-\d{2}", k)

    hariini = datetime.now()
    hasil = []

    for dstr in tgl:
        do = datetime.strptime(dstr, "%Y-%m-%d")

        hs = (hariini - do).days

        fd = do.strftime("%d-%m-%Y")

        hasil.append(f"{fd}{do.time()} selisih {hs} hari")

    return hasil

teks = """
Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan
nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki
Hajar Dewantara (1889-05-02).
"""
hasil = nemutgldnhtg(teks)
for h in hasil:
    print(h)
```

Gambar 9.10: Program mencari seluruh teks yang berbentuk tanggal

Program pada Latihan 9.5 dirancang untuk mengekstraksi tanggal berformat YYYY-MM-DD dari sebuah teks menggunakan pola RegEx \d{4}-\d{2}-\d{2}. Setelah tanggal ditemukan, data tersebut dikonversi menjadi objek datetime agar bisa dihitung selisih harinya terhadap waktu saat ini (datetime.now()). Sebagai hasil akhir, program menampilkan kembali tanggal tersebut dalam format DD-MM-YYYY lengkap dengan informasi selisih hari yang telah dihitung.

LATIHAN 9.6

```python
import re
import random
import string

def gpw(length=8):
    huruf = string.ascii_letters + string.digits
    return ''.join(random.choice(huruf) for _ in range(length))

def egpw(text):
    # Cari semua email
    emil = re.findall(r'\b[\w\.-]+@[\w\.-]+\.\w+\b', text)

    results = []
    for email in emil:
        # Ambil username sebelum '@'
        username = email.split('@')[0]

        # Generate password random 8 karakter
        password = gpw()

        # Simpan hasil
        results.append(f"{email} username: {username} , password: {password}")

    return results

# Contoh penggunaan
teks = """
Berikut adalah daftar email dan nama pengguna dari mailing list:
anton@mail.com dimiliki oleh antonius
budi@gmail.co.id dimiliki oleh budi anwari
slamet@getnada.com dimiliki oleh slamet slumut
matahari@tokopedia.com dimiliki oleh toko matahari
"""

hasil = egpw(teks)
for h in hasil:
    print(h)
```

Gambar 9.11: Program membuat password

Latihan 9.6 berfokus pada ekstraksi alamat email dari sebuah paragraf menggunakan pola RegEx \b[\w\.-]+@[\w\.-]+\.\w+\b. Setelah sistem berhasil mengidentifikasi email berdasarkan struktur karakter dan domainnya, program akan mengambil bagian *username* (teks sebelum simbol @). Selanjutnya, setiap *username* akan dipasangkan dengan kata sandi acak 8 karakter yang dihasilkan melalui fungsi random.choice() dari pustaka string. Hasil akhirnya adalah daftar lengkap berisi email, nama pengguna, dan kata sandi baru yang disusun secara efisien tanpa proses pembedahan teks manual.